

1.1 Introduction

The Student Learning Management System (SLMS) is designed to support academic activities between students and lecturers. The system enables lecturers to manage course materials and assignments, while allowing students to access learning resources, submit assignments, and view grades

The overall system design is represented using Use Case and Class diagrams to illustrate system functionality and object-oriented structure

1.2 Use Case Design

1.2.1 Actors

The SLMS consists of two main actors:

- **Student** – A registered user who enrolls in courses, views materials, submits assignments, and views grades.
- **Lecturer** – A user responsible for managing course materials, creating assignments, and grading student submissions.

1.2.2 Use Case Description

The Use Case Diagram illustrates how users interact with the SLMS system.

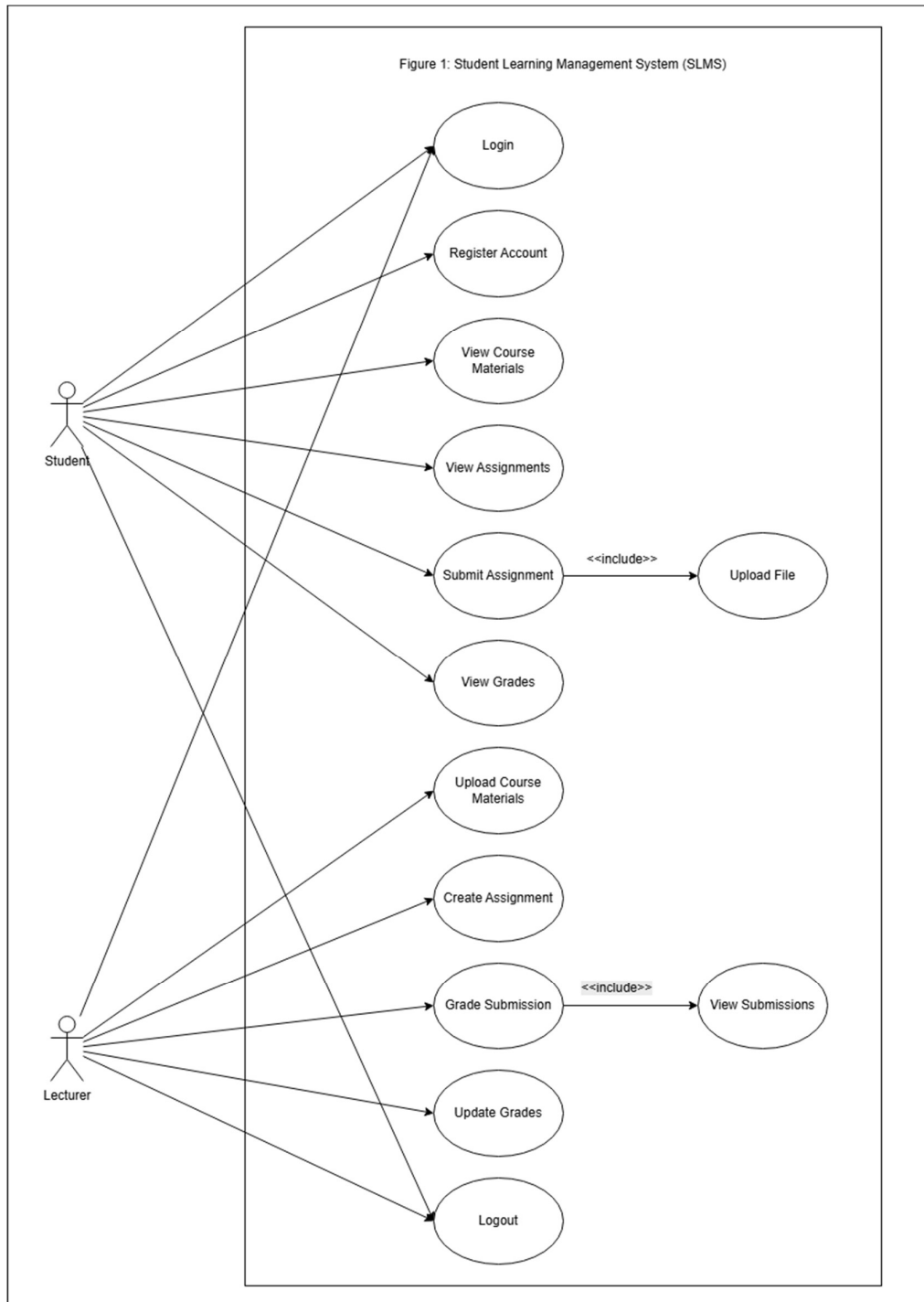
Student Functionalities:

- Register and login
- View course materials
- View assignments
- Submit assignments
- View grades

Lecturer Functionalities:

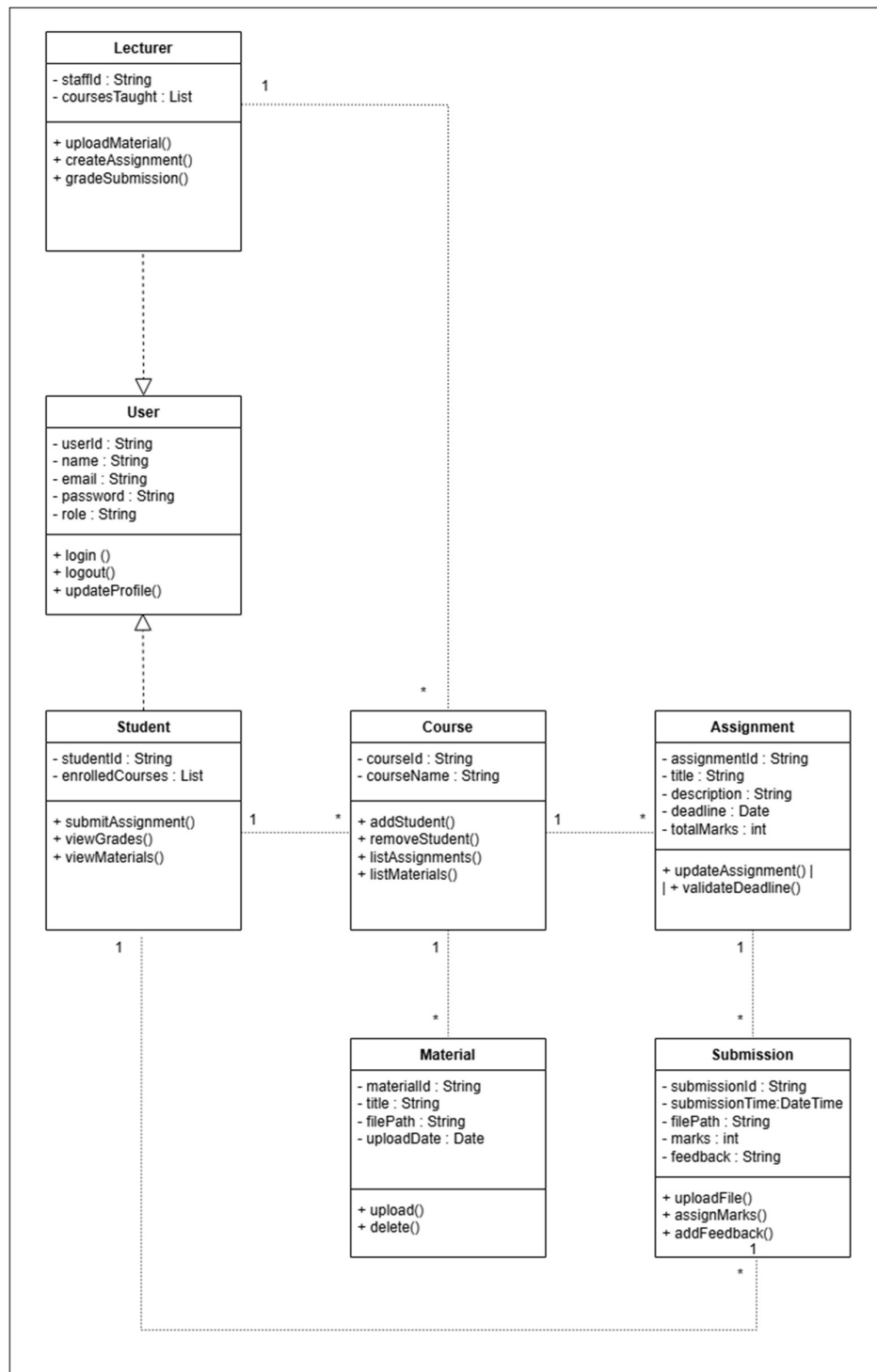
- Login
- Upload course materials
- Create and update assignments
- View student submissions
- Grade and publish results

Use Case



The Use Case Diagram illustrates the interaction between two primary actors, Student and Instructor, with the SLMS system. Students can register, log in, view materials, submit assignments, and view grades. Instructors can upload materials, create assignments, view submissions, and grade student work. Certain use cases such as "Submit Assignment" include "Upload File" to represent dependent functionality. The diagram defines system boundaries and actor interactions clearly.

Class Diagram



The Class Diagram represents the object-oriented structure of the SLMS. The system follows inheritance where Student and Instructor extend the User class. A Course aggregates Assignments and Materials. Each Assignment can have multiple Submissions. This structure promotes modularity, reusability, and scalability by clearly separating responsibilities across classes.

5. Define Function and Methods Modularity

5.1 Introduction

In the SLMS system, functions and methods are designed using modular programming principles. A function is a block of code that performs a specific task. A method is a function that belongs to a class in object-oriented programming. Modularity refers to organizing these functions and methods into independent modules where each module is responsible for one specific functionality.

5.2 Function-Level Modularity

Each function in the SLMS performs only one clear responsibility. This follows the **Single Responsibility Principle (SRP)**.

Examples of function-level modularity:

- `loginUser()` → validates user credentials only.
- `submitAssignment()` → handles assignment submission only.
- `calculateFinalGrade()` → computes final grades only.
- `uploadMaterial()` → manages file upload for learning materials only.

No function performs multiple unrelated tasks. This avoids overly complex methods and improves readability.

5.3 Module-Level Modularity

The SLMS system is divided into logical modules. Each module groups related functions together and separates them from other functionalities.

1. Authentication Module

Responsible for **user login, logout, and verification**.

Functions:

- registerUser()
- loginUser()
- logoutUser()
- verifyTwoFactorCode()

2. User Management Module

Responsible for **handling user account information**.

Functions:

- createUser()
- updateUser()
- deleteUser()
- getUserById()

3. Assignment Module

Responsible for **creating and managing assignments**.

Functions:

- createAssignment()
- updateAssignment()
- deleteAssignment()
- validateDeadline()

4. Submission Module

Responsible for **handling student assignment submissions**.

Functions:

- submitAssignment()
- checkLateSubmission()
- getSubmissionsByAssignment()

5. Grading Module

Responsible for **evaluating and grading submissions**.

Functions:

- assignMarks()
- addFeedback()
- calculateFinalGrade()
- publishGrades()

6. Material Module

Responsible for **managing learning materials**.

Functions:

- uploadMaterial()
- deleteMaterial()
- listMaterials()